

Google Play Store DataAnalysis

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Data Collection

```
In [2]: df = pd.read_csv(r"C:\Users\HP\Documents\Nareshit\Pandas\EDA\dataset\Google-Playsto
df.head()
```

Out[2]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0

5 rows × 24 columns



Data Cleaning

```
In [3]: df.head(5)
```

Out[3]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0

5 rows × 24 columns



In [4]: `df.shape`

Out[4]: (2312944, 24)

```
In [5]: df.isnull().sum()
# mean , median , mode , dropna , ffill , bfill
# if the null <5% then use dropna
# if the column category -> mode
# if the column has extreme values(outliers) -> median
# if the columns has numeric (normal values) -> mean
# if the column is time based then -> ffill/bfill
```

```
Out[5]: App Name          5
        App Id           0
        Category         0
        Rating           22883
        Rating Count     22883
        Installs         107
        Minimum Installs 107
        Maximum Installs 0
        Free             0
        Price            0
        Currency         135
        Size             196
        Minimum Android   6530
        Developer Id      33
        Developer Website 760835
        Developer Email   31
        Released          71053
        Last Updated      0
        Content Rating    0
        Privacy Policy    420953
        Ad Supported      0
        In App Purchases  0
        Editors Choice    0
        Scraped Time      0
        dtype: int64
```

```
In [6]: df.dtypes
```

```
Out[6]: App Name          object
        App Id           object
        Category         object
        Rating           float64
        Rating Count     float64
        Installs         object
        Minimum Installs float64
        Maximum Installs int64
        Free             bool
        Price            float64
        Currency         object
        Size             object
        Minimum Android   object
        Developer Id      object
        Developer Website object
        Developer Email   object
        Released          object
        Last Updated      object
        Content Rating    object
        Privacy Policy    object
        Ad Supported      bool
        In App Purchases bool
        Editors Choice    bool
        Scraped Time      object
        dtype: object
```

```
In [7]: df.fillna(df['Rating'].mean(), inplace = True)
```

```
In [8]: df['Rating'].isnull().sum()
```

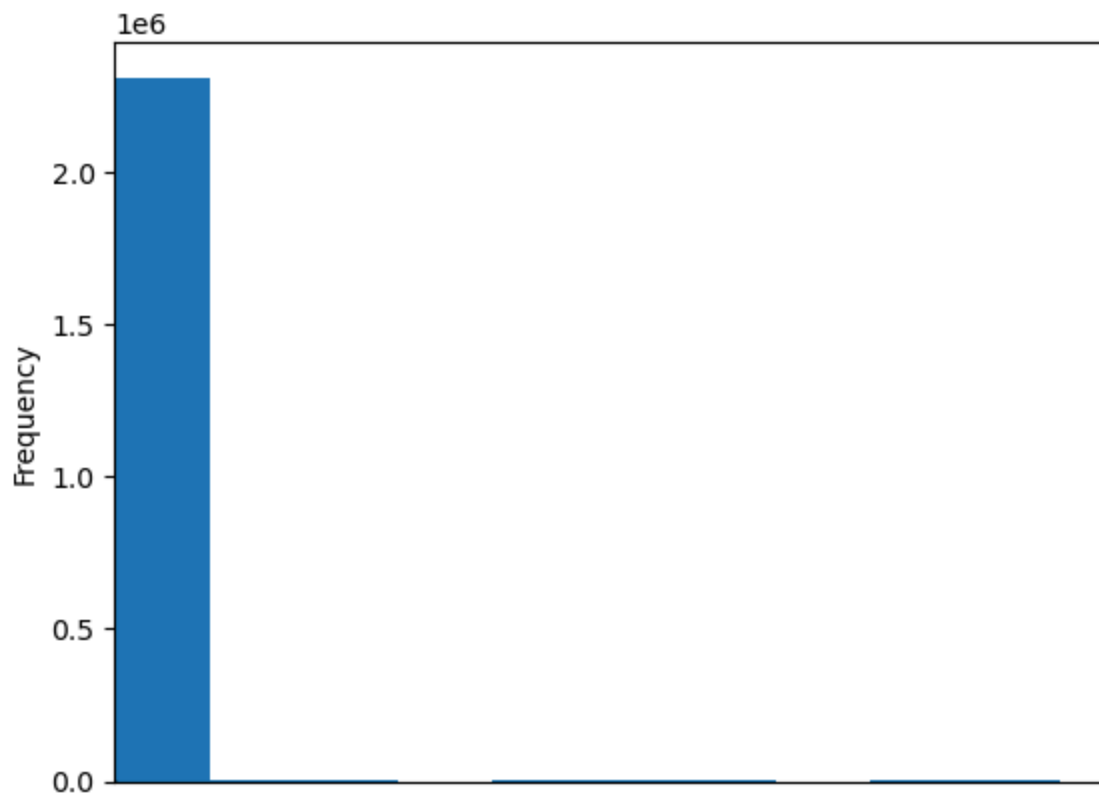
```
Out[8]: np.int64(0)
```

```
In [9]: # Before filling null value  
df['Rating Count'].skew()  
df['Rating Count'].plot(kind='hist')
```

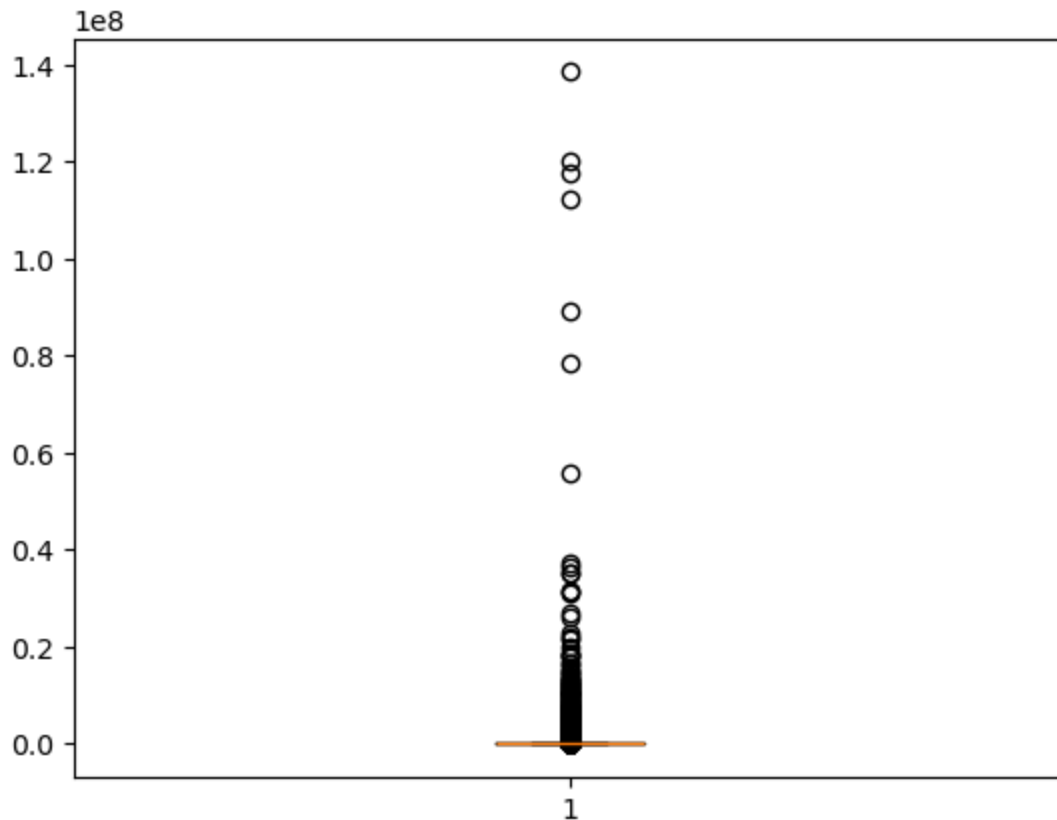
```
Out[9]: <Axes: ylabel='Frequency'>
```

```
In [10]: df.fillna(df['Rating Count'].median(), inplace = True)
```

```
In [11]: plt.boxplot(df['Rating'])  
plt.show()
```



```
In [12]: plt.boxplot(df['Rating Count'])  
plt.show()
```



```
In [13]: df.columns
```

```
Out[13]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
               'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
               'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
               'Developer Email', 'Released', 'Last Updated', 'Content Rating',
               'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
               'Scraped Time'],
              dtype='object')
```

```
In [14]: df.dropna(subset = ['App Name', 'Installs', 'Minimum Installs', 'Currency', 'Size'])
```

```
In [15]: df.isnull().sum()
```

```
Out[15]: App Name      0
App Id      0
Category    0
Rating      0
Rating Count 0
Installs    0
Minimum Installs 0
Maximum Installs 0
Free        0
Price       0
Currency    0
Size        0
Minimum Android 0
Developer Id 0
Developer Website 0
Developer Email 0
Released    0
Last Updated 0
Content Rating 0
Privacy Policy 0
Ad Supported 0
In App Purchases 0
Editors Choice 0
Scraped Time 0
dtype: int64
```

Looking for Duplicate

```
In [16]: df['App Name'].duplicated().any()
```

```
Out[16]: np.True_
```

```
In [17]: df['App Name'].value_counts()
```

```
Out[17]: App Name
Tic Tac Toe      382
Calculator      261
Flashlight      256
BMI Calculator   201
Age Calculator   190
...
STMIK Mercusuar - Aditya Rachman      1
Lero TOEFL Recorder + Timer           1
大俠客—熱血歸來                        1
ORU Online                           1
Gluten-free Recipes                   1
Name: count, Length: 2177944, dtype: int64
```

```
In [18]: # select Tic Tac Toe app

df[df['App Name']== 'Tic Tac Toe' ]
```

Out[18]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minir Ins
3918	Tic Tac Toe	com.dev.atte.tic_tac_toe	Puzzle	0.0	0.0	10+	
13758	Tic Tac Toe	com.md.tictactoe	Board	0.0	0.0	10+	
25351	Tic Tac Toe	com.sthirumuru.tictactoenew	Board	4.2	11.0	5,000+	50
28142	Tic Tac Toe	com.fsk00x.TicTacToe	Board	0.0	0.0	500+	5
30962	Tic Tac Toe	net.snatchdreams.tictactoe	Puzzle	0.0	0.0	10+	
...	...	...	...	...	...	...	
2272773	Tic Tac Toe	com.tictactoe.MobileAppsPro	Board	3.8	9269.0	500,000+	5000
2281376	Tic Tac Toe	com.erjainam.tictactoe.release	Board	0.0	0.0	50+	
2282023	Tic Tac Toe	com.swapworld.tic_tac_toe	Board	4.7	7.0	10+	
2299121	Tic Tac Toe	com.infuse.tictactoe	Board	4.8	20.0	100+	1
2310375	Tic Tac Toe	com.justmailtoavi.avinashk.threet	Puzzle	4.7	10.0	100+	1

382 rows × 24 columns



Observation

- Although all of our apps have the same name, they are differentiated by their unique app identifiers.

In [19]: `df.columns`

```
Out[19]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
              'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
              'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
              'Developer Email', 'Released', 'Last Updated', 'Content Rating',
              'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
              'Scraped Time'],
              dtype='object')
```

```
In [20]: df['Installs'].unique()
```

```
Out[20]: array(['10+', '5,000+', '50+', '100+', '1,000+', '500+', '50,000+',
              '10,000+', '1+', '500,000+', '100,000+', '5+', '10,000,000+',
              '1,000,000+', '5,000,000+', '0+', '100,000,000+',
              np.float64(2.203151531771424), '50,000,000+', '1,000,000,000+',
              '500,000,000+', '5,000,000,000+', '10,000,000,000+'], dtype=object)
```

```
In [21]: df['Installs'] = df['Installs'].str.replace('+','', regex=False).str.replace(',','',
```

```
In [22]: import pandas as pd
df['Installs'] = pd.to_numeric(df['Installs'], errors='coerce')
```

```
In [23]: df['Installs'].unique()
```

```
Out[23]: array([1.e+01, 5.e+03, 5.e+01, 1.e+02, 1.e+03, 5.e+02, 5.e+04, 1.e+04,
              1.e+00, 5.e+05, 1.e+05, 5.e+00, 1.e+07, 1.e+06, 5.e+06, 0.e+00,
              1.e+08,      nan, 5.e+07, 1.e+09, 5.e+08, 5.e+09, 1.e+10])
```

```
In [24]: # print(1.e+01)
```

Dealing with Size columns

```
In [25]: df['Size'].unique()
```

```
Out[25]: array(['10M', '2.9M', '3.7M', ..., '405M', '3.2k', '512M'], dtype=object)
```

```
In [26]: df['Size'].value_counts()
```

```
Out[26]: Size
Varies with device    74777
11M                  62157
12M                  56080
13M                  48034
14M                  45211
...
470M                   1
910M                   1
959M                   1
3.2k                   1
512M                   1
Name: count, Length: 1658, dtype: int64
```

```
In [27]: df['Size'] = df['Size'].str.replace('M', '000000')
df['Size'] = df['Size'].str.replace('K', '000')
df['Size'] = df['Size'].str.replace('G', '000000000')
```



```
In [28]: df['Size'] = pd.to_numeric(df['Size'], errors='coerce')
```

```
In [29]: df['Size'].unique()
```

```
Out[29]: array([1.00e+07, 2.90e+00, 3.70e+00, 1.80e+00, 6.20e+00, 4.60e+07,
                2.50e+00, 1.60e+07, 1.30e+00, 3.50e+00, 5.10e+07, 2.70e+00,
                7.60e+00, 1.50e+07, 2.90e+07, 1.20e+07, 2.00e+00, 7.00e+07,
                1.10e+07, 2.40e+07, 1.40e+00, 2.00e+07, 2.40e+00, 2.50e+07,
                2.10e+07, 3.60e+07, 6.10e+00, 2.60e+00, 8.90e+00, 3.90e+00,
                3.00e+07, 2.60e+07, 4.90e+00, 4.30e+00, 4.80e+00, 4.60e+00,
                2.80e+00, 3.80e+07, 5.10e+00, 1.47e+08, 3.30e+07, 2.20e+00,
                7.10e+00, 2.70e+07, 1.80e+07, 7.70e+00, 3.20e+07, 1.90e+07,
                3.90e+07, 3.10e+07, 1.20e+00, 3.10e+00, 3.20e+00, 4.20e+07,
                2.20e+07, 4.10e+00, 5.00e+07, 5.50e+00,          nan, 8.40e+00,
                4.10e+07, 4.90e+07, 7.00e+00, 4.70e+00, 4.30e+07, 5.40e+00,
                3.00e+00, 1.60e+00, 2.10e+00, 3.70e+07, 3.60e+00, 9.00e+00,
                9.90e+00, 6.70e+07, 2.80e+07, 7.90e+00, 3.30e+00, 6.00e+00,
                5.70e+00, 9.80e+00, 5.00e+00, 1.70e+00, 7.40e+00, 1.90e+00,
                3.80e+00, 6.40e+00, 8.10e+00, 6.50e+00, 4.40e+00, 4.50e+07,
                7.20e+00, 1.30e+07, 6.90e+00, 6.00e+07, 6.60e+00, 4.00e+00,
                1.70e+07, 4.20e+00, 5.80e+07, 9.30e+07, 9.70e+00, 2.30e+07,
                5.20e+07, 5.60e+00, 4.80e+07, 1.50e+00, 8.50e+00, 5.40e+07,
                6.20e+07, 1.33e+08, 6.40e+07, 1.52e+08, 1.36e+08, 9.50e+00,
                4.50e+00, 4.00e+07, 8.00e+00, 2.30e+00, 3.50e+07, 8.30e+07,
                7.30e+00, 6.70e+00, 9.40e+00, 6.90e+07, 7.50e+00, 8.40e+07,
                5.90e+00, 6.30e+00, 3.40e+07, 6.30e+07, 3.40e+00, 5.70e+07,
                1.40e+07, 8.60e+07, 9.20e+00, 1.42e+08, 6.80e+00, 9.10e+00,
                9.60e+00, 4.40e+07, 1.00e+00, 5.30e+07, 9.10e+07, 6.10e+07,
                5.20e+00, 5.30e+00, 8.30e+00, 8.20e+00, 7.90e+07, 9.30e+00,
                4.70e+07, 1.07e+08, 8.60e+00, 9.50e+07, 5.60e+07, 5.80e+00,
                7.10e+07, 7.80e+07, 7.80e+00, 5.90e+07, 7.20e+07, 8.70e+00,
                7.50e+07, 1.51e+08, 1.45e+08, 8.80e+00, 8.20e+07, 1.00e+01,
                6.60e+07, 1.01e+08, 7.60e+07, 9.60e+07, 1.39e+08, 1.25e+08,
                1.11e+08, 1.10e+00, 1.16e+08, 9.40e+07, 9.90e+07, 8.90e+07,
                5.50e+07, 8.50e+07, 9.80e+07, 1.22e+08, 8.80e+07, 7.70e+07,
                1.06e+08, 8.70e+07, 9.70e+07, 2.01e+08, 6.80e+07, 1.18e+08,
                7.40e+07, 7.30e+07, 1.44e+08, 2.50e+08, 8.10e+07, 1.12e+08,
                6.50e+07, 1.02e+08, 1.05e+08, 1.00e+08, 1.13e+08, 5.31e+08,
                1.20e+08, 1.41e+08, 1.30e+08, 9.20e+07, 9.00e+07, 1.50e+08,
                8.00e+07, 1.56e+08, 1.21e+08, 1.43e+08, 1.37e+08, 1.59e+08,
                4.22e+08, 1.49e+08, 1.34e+08, 1.15e+08, 1.28e+08, 2.30e+08,
                1.03e+08, 1.46e+08, 1.14e+08, 1.38e+08, 1.17e+08, 1.10e+08,
                1.58e+08, 1.48e+08, 1.27e+08, 1.09e+08, 1.53e+08, 1.62e+08,
                1.19e+08, 1.32e+08, 1.65e+08, 1.04e+08, 1.40e+08, 1.23e+08,
                1.24e+08, 1.73e+08, 2.93e+08, 1.74e+08, 1.57e+08, 1.35e+08,
                1.67e+08, 1.84e+08, 3.35e+08, 2.28e+08, 1.88e+08, 1.29e+08,
                1.54e+08, 1.60e+08, 1.64e+08, 1.08e+08, 5.10e+08, 1.31e+08,
                1.76e+08, 2.43e+08, 2.06e+08, 1.66e+08, 2.48e+08, 1.94e+08,
                2.10e+08, 1.87e+08, 2.31e+08, 1.77e+08, 2.02e+08, 1.80e+08,
                1.85e+08, 1.98e+08, 7.22e+08, 1.55e+08, 1.89e+08, 1.81e+08,
                3.37e+08, 1.26e+08, 2.09e+08, 2.87e+08, 1.83e+08, 1.70e+08,
                2.80e+08, 1.91e+08, 1.78e+08, 2.16e+08, 2.24e+08, 3.64e+08,
                1.72e+08, 2.51e+08, 1.90e+08, 3.39e+08, 1.96e+08, 3.43e+08,
                1.97e+08, 2.57e+08, 1.95e+08, 2.00e+08, 2.12e+08, 1.99e+08,
                1.71e+08, 1.82e+08, 1.79e+08, 4.67e+08, 9.28e+08, 2.03e+08,
                5.09e+08, 2.18e+08, 2.66e+08, 1.75e+08, 2.56e+08, 2.34e+08,
                1.63e+08, 1.61e+08, 1.93e+08, 2.15e+08, 7.06e+08, 3.32e+08,
                5.65e+08, 2.98e+08, 2.13e+08, 2.75e+08, 3.08e+08, 3.20e+08,
                1.68e+08, 2.85e+08, 2.35e+08, 2.07e+08, 2.04e+08, 2.33e+08,
                2.65e+08, 1.69e+08, 2.54e+08, 6.56e+08, 2.84e+08, 1.92e+08,
```

1.86e+08, 2.69e+08, 2.49e+08, 2.38e+08, 3.38e+08, 2.55e+08,
2.99e+08, 2.52e+08, 2.27e+08, 2.62e+08, 3.98e+08, 2.21e+08,
2.88e+08, 2.39e+08, 3.06e+08, 4.48e+08, 2.61e+08, 3.24e+08,
2.14e+08, 5.01e+08, 2.77e+08, 3.60e+08, 2.37e+08, 2.82e+08,
9.85e+08, 2.68e+08, 4.12e+08, 5.08e+08, 2.19e+08, 2.58e+08,
2.76e+08, 3.33e+08, 6.18e+08, 2.53e+08, 8.41e+08, 3.13e+08,
5.76e+08, 6.90e+08, 2.25e+08, 3.04e+08, 8.32e+08, 2.97e+08,
6.03e+08, 4.90e+08, 2.26e+08, 9.54e+08, 2.90e+08, 2.44e+08,
4.97e+08, 2.64e+08, 3.14e+08, 2.95e+08, 6.43e+08, 2.05e+08,
4.56e+08, 4.93e+08, 2.70e+08, 3.21e+08, 4.14e+08, 9.25e+08,
2.08e+08, 2.45e+08, 2.11e+08, 2.74e+08, 3.30e+08, 3.91e+08,
6.45e+08, 6.52e+08, 4.20e+08, 2.17e+08, 3.48e+08, 2.94e+08,
4.47e+08, 2.72e+08, 3.83e+08, 9.19e+08, 2.41e+08, 2.20e+08,
9.04e+08, 2.22e+08, 2.46e+08, 5.95e+08, 4.85e+08, 3.19e+08,
9.00e+08, 3.61e+08, 3.45e+08, 2.47e+08, 8.95e+08, 4.06e+08,
3.09e+08, 2.86e+08, 3.11e+08, 3.22e+08, 3.28e+08, 3.03e+08,
3.31e+08, 3.12e+08, 2.79e+08, 8.18e+08, 3.23e+08, 6.49e+08,
3.17e+08, 2.40e+08, 2.29e+08, 3.00e+08, 2.42e+08, 3.59e+08,
4.95e+08, 9.35e+08, 2.60e+08, 5.64e+08, 2.91e+08, 3.63e+08,
3.75e+08, 5.18e+08, 7.62e+08, 4.91e+08, 3.72e+08, 6.76e+08,
6.84e+08, 5.79e+08, 5.50e+08, 4.07e+08, 5.91e+08, 3.44e+08,
2.32e+08, 4.18e+08, 2.63e+08, 3.46e+08, 4.60e+08, 2.67e+08,
8.43e+08, 2.83e+08, 3.53e+08, 7.84e+08, 2.23e+08, 3.85e+08,
2.36e+08, 5.67e+08, 8.68e+08, 5.78e+08, 4.45e+08, 3.51e+08,
2.81e+08, 3.10e+08, 2.92e+08, 8.78e+08, 2.89e+08, 5.44e+08,
4.26e+08, 6.81e+08, 3.47e+08, 4.54e+08, 3.27e+08, 6.06e+08,
3.52e+08, 9.40e+08, 3.90e+08, 2.71e+08, 7.65e+08, 5.66e+08,
6.64e+08, 4.11e+08, 6.80e+08, 3.05e+08, 5.14e+08, 4.09e+08,
2.73e+08, 6.39e+08, 2.96e+08, 9.63e+08, 4.21e+08, 4.57e+08,
3.87e+08, 3.81e+08, 3.18e+08, 9.53e+08, 4.37e+08, 6.11e+08,
3.71e+08, 3.96e+08, 3.77e+08, 8.96e+08, 4.15e+08, 4.68e+08,
5.90e+08, 3.55e+08, 5.93e+08, 4.24e+08, 6.46e+08, 5.27e+08,
3.66e+08, 4.88e+08, 3.01e+08, 3.97e+08, 7.45e+08, 4.65e+08,
5.03e+08, 5.70e+08, 8.69e+08, 3.29e+08, 3.73e+08, 3.40e+08,
4.08e+08, 3.02e+08, 7.05e+08, 3.69e+08, 4.40e+08, 6.79e+08,
5.68e+08, 5.19e+08, 5.80e+08, 9.77e+08, 9.59e+08, 9.81e+08,
2.59e+08, 5.26e+08, 3.79e+08, 6.23e+08, 6.19e+08, 2.78e+08,
5.21e+08, 3.15e+08, 8.44e+08, 8.89e+08, 9.96e+08, 3.65e+08,
4.31e+08, 5.81e+08, 3.34e+08, 9.62e+08, 5.41e+08, 7.99e+08,
3.42e+08, 3.70e+08, 5.32e+08, 3.82e+08, 4.42e+08, 7.37e+08,
8.66e+08, 3.94e+08, 9.14e+08, 6.91e+08, 7.25e+08, 4.61e+08,
6.33e+08, 3.49e+08, 7.00e+08, 3.56e+08, 7.20e+08, 7.44e+08,
4.29e+08, 5.33e+08, 8.10e+08, 4.04e+08, 3.54e+08, 7.69e+08,
6.92e+08, 8.39e+08, 8.30e+08, 7.12e+08, 4.70e+08, 9.10e+08,
4.05e+08, 5.12e+08])

```
In [30]: df['Size'].dtypes
```

```
Out[30]: dtype('float64')
```

```
In [31]: df.columns
```

```
Out[31]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
               'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
               'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
               'Developer Email', 'Released', 'Last Updated', 'Content Rating',
               'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
               'Scraped Time'],
              dtype='object')
```

```
In [32]: df['Minimum Android'].unique()
```

```
Out[32]: array(['7.1 and up', '5.0 and up', '4.0.3 and up', '4.1 and up',
               '6.0 and up', '4.4 and up', '4.0 and up', '4.2 and up',
               '2.1 and up', '7.0 and up', '2.2 and up', '2.3 and up',
               '4.3 and up', '5.1 and up', np.float64(2.203151531771424),
               '1.6 and up', '3.0 and up', 'Varies with device', '8.0 and up',
               '2.3.3 and up', '4.4W and up', '3.2 and up', '1.5 and up',
               '4.0.3 - 7.1.1', '3.1 and up', '2.0 and up', '1.0 and up',
               '1.1 and up', '2.0.1 and up', '4.4 - 6.0', '2.1 - 4.4',
               '4.0 - 5.0', '2.0 - 2.3.4', '2.2 - 4.4', '4.1 - 7.0', '4.1 - 6.0',
               '4.0 - 4.4W', '4.0 - 4.4', '4.0.3 - 7.0', '2.3 - 5.0',
               '4.0 - 7.1.1', '4.1 - 7.1.1', '4.0 - 7.0', '4.1 - 8.0',
               '4.0 - 6.0', '2.3 - 7.0', '4.0 - 8.0', '2.3 - 6.0', '2.1 - 2.3.4',
               '3.0 - 4.1.1', '2.2 - 3.0', '2.0 - 8.0', '4.0.3 - 8.0',
               '3.0 - 4.4W', '4.4 - 7.1.1', '4.4 - 8.0', '4.0.3 - 6.0',
               '1.6 - 4.0.4', '8.0', '2.3 - 5.1', '2.1 - 7.1.1', '2.3 - 4.4W',
               '4.4 - 7.0', '6.0 - 7.1.1', '2.3 - 4.4', '2.2 - 4.0.4',
               '2.1 - 4.1.1', '3.0 - 8.0', '3.0 - 5.1', '2.1 - 6.0', '6.0 - 8.0',
               '5.0 - 8.0', '4.1 - 5.1', '3.2 - 4.4', '2.1 - 5.0', '4.1 - 4.3',
               '5.0 - 6.0', '5.1 - 7.1.1', '2.2 - 5.1', '4.1 - 5.0',
               '2.3.3 - 4.4', '2.2 - 5.0', '2.2', '4.1 - 4.4', '1.6 - 5.1',
               '4.2 - 7.1.1', '2.1 - 5.1', '1.6 - 4.4W', '4.3 - 4.4W',
               '3.0 - 4.4', '2.1 - 3.1', '2.3 - 7.1.1', '1.6 - 4.0.2',
               '4.0 - 4.0.4', '4.2 - 4.4W', '3.0 - 7.1.1', '3.2 - 7.1.1',
               '3.0 - 6.0', '2.2 - 4.4W', '4.0.3 - 4.4', '4.0.3 - 5.1', '6.0',
               '4.3', '1.1 - 4.4', '2.3.3 - 8.0', '2.3.3 - 5.0', '2.2 - 6.0',
               '3.0 - 5.0', '2.3.3 - 2.3.4', '1.5 - 2.1', '2.2 - 3.2',
               '5.0 - 7.1.1', '2.1 - 3.2', '4.2 - 4.2.2', '2.3.3 - 5.1',
               '2.2 - 4.1.1', '1.6 - 4.4', '4.2 - 4.3', '4.0 - 5.1', '1.6 - 7.0',
               '3.0 - 7.0', '1.0 - 6.0', '2.3.3 - 4.0.4', '2.3 - 3.2',
               '4.3 - 8.0', '2.3 - 8.0', '3.2 - 6.0', '2.3.3 - 7.1.1',
               '4.2 - 8.0', '2.2 - 4.3', '3.2 - 4.1.1', '4.4 - 5.1',
               '2.3 - 4.1.1', '2.1 - 4.4W', '4.4', '2.3.3 - 4.4W', '4.1 - 4.4W',
               '1.6 - 7.1.1', '2.2 - 4.2.2', '4.3 - 5.1', '4.0.3 - 4.2.2',
               '2.2 - 2.3.4', '4.3 - 4.4', '4.2 - 6.0', '4.4 - 4.4W',
               '2.3 - 4.0.2', '3.2 - 5.1', '7.0', '3.0 - 3.2', '1.6 - 4.2.2',
               '5.1', '1.5 - 3.2', '2.2 - 8.0', '1.6 - 2.1', '2.3.3 - 6.0'],
              dtype=object)
```

```
In [33]: df['Minimum Android'].value_counts()
```

```
Out[33]: Minimum Android
4.1 and up      604465
5.0 and up      396998
4.4 and up      390311
4.0.3 and up    180482
4.0 and up      153441
...
5.1              1
1.5 - 3.2        1
2.2 - 8.0        1
1.6 - 2.1        1
2.3.3 - 6.0      1
Name: count, Length: 155, dtype: int64
```

```
In [34]: df['Content Rating'].unique()
```

```
Out[34]: array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
                'Adults only 18+', 'Unrated'], dtype=object)
```

```
In [35]: df['Content Rating']
```

```
Out[35]: 0      Everyone
1      Everyone
2      Everyone
3      Everyone
4      Everyone
...
2312939      Teen
2312940      Everyone
2312941      Everyone
2312942      Everyone
2312943      Everyone
Name: Content Rating, Length: 2312944, dtype: object
```

```
In [36]: df['Released']
```

```
Out[36]: 0      Feb 26, 2020
1      May 21, 2020
2      Aug 9, 2019
3      Sep 10, 2018
4      Feb 21, 2020
...
2312939      2.203152
2312940      Jan 17, 2018
2312941      Aug 19, 2018
2312942      Aug 1, 2016
2312943      Aug 9, 2019
Name: Released, Length: 2312944, dtype: object
```

```
In [37]: df['Last Updated']
```

```

Out[37]: 0          Feb 26, 2020
        1          May 06, 2021
        2          Aug 19, 2019
        3          Oct 13, 2018
        4          Nov 12, 2018
        ...
        2312939     Jun 01, 2021
        2312940     Feb 02, 2018
        2312941     Aug 19, 2018
        2312942     May 05, 2021
        2312943     Aug 19, 2019
Name: Last Updated, Length: 2312944, dtype: object

```

- Lets clean the content rating
- Mature 17+ -----> to Adults
- Adults only 18 -----> to Adults
- Everyone 10+ -----> to Teen
- Unrated -----> Everyone

```

In [38]: df['Content Rating'] = df['Content Rating'].replace('Unrated', 'Everyone')
df['Content Rating'] = df['Content Rating'].replace('Mature 17+', 'Adults')
df['Content Rating'] = df['Content Rating'].replace('Everyone 10+', 'Teen')
df['Content Rating'] = df['Content Rating'].replace('Adults only 18+', 'Adults')

```

```

In [39]: df['Content Rating'].unique()

```

```

Out[39]: array(['Everyone', 'Teen', 'Adults'], dtype=object)

```

```

In [40]: df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2312944 entries, 0 to 2312943
Data columns (total 24 columns):
#   Column                Dtype
---  -
0   App Name              object
1   App Id                object
2   Category              object
3   Rating                float64
4   Rating Count          float64
5   Installs              float64
6   Minimum Installs      float64
7   Maximum Installs      int64
8   Free                  bool
9   Price                 float64
10  Currency              object
11  Size                  float64
12  Minimum Android        object
13  Developer Id           object
14  Developer Website      object
15  Developer Email        object
16  Released               object
17  Last Updated           object
18  Content Rating         object
19  Privacy Policy         object
20  Ad Supported           bool
21  In App Purchases       bool
22  Editors Choice         bool
23  Scraped Time           object
dtypes: bool(4), float64(6), int64(1), object(13)
memory usage: 361.8+ MB

```

```

In [41]: df.columns
# df['Rating count'].unique()

```

```

Out[41]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
               'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
               'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
               'Developer Email', 'Released', 'Last Updated', 'Content Rating',
               'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
               'Scraped Time'],
              dtype='object')

```

```

In [42]: df['Rating Count'].unique()

```

```

Out[42]: array([0.0000e+00, 6.4000e+01, 5.0000e+00, ..., 8.7553e+04, 7.5960e+04,
               7.8351e+04])

```

```

In [43]: df['Rating Count'].max()

```

```

Out[43]: 138557570.0

```

```

In [44]: def categorize_rating(x):
          if x < 10000:
              return 'less than 10k'
          elif 10000 <= x <= 500000:

```

```

        return '10k to 500k'
    else:
        return 'more than 500k'

df['Rating Category'] = df['Rating Count'].apply(categorize_rating)

df['Rating Category']

```

```

Out[44]: 0          less than 10k
        1          less than 10k
        2          less than 10k
        3          less than 10k
        4          less than 10k
        ...
        2312939      10k to 500k
        2312940      less than 10k
        2312941      less than 10k
        2312942      less than 10k
        2312943      less than 10k
        Name: Rating Category, Length: 2312944, dtype: object

```

categorical Analysis

- Q1.According to the count, which 10 categories are there in the Play Store?

```

In [45]: df.columns

```

```

Out[45]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
               'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
               'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
               'Developer Email', 'Released', 'Last Updated', 'Content Rating',
               'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
               'Scraped Time', 'Rating Category'],
              dtype='object')

```

```

In [46]: df['Category'].unique()

```

```

Out[46]: array(['Adventure', 'Tools', 'Productivity', 'Communication', 'Social',
               'Libraries & Demo', 'Lifestyle', 'Personalization', 'Racing',
               'Maps & Navigation', 'Travel & Local', 'Food & Drink',
               'Books & Reference', 'Medical', 'Puzzle', 'Entertainment',
               'Arcade', 'Auto & Vehicles', 'Photography', 'Health & Fitness',
               'Education', 'Shopping', 'Board', 'Music & Audio', 'Sports',
               'Beauty', 'Business', 'Educational', 'Finance', 'News & Magazines',
               'Casual', 'Art & Design', 'House & Home', 'Card', 'Events',
               'Trivia', 'Weather', 'Strategy', 'Word', 'Video Players & Editors',
               'Action', 'Simulation', 'Music', 'Dating', 'Role Playing',
               'Casino', 'Comics', 'Parenting'], dtype=object)

```

```

In [47]: top_category = df.Category.value_counts().reset_index()

```



```
In [48]: top_category.head(10)
```

```
Out[48]:
```

	Category	count
0	Education	241090
1	Music & Audio	154906
2	Tools	143988
3	Business	143771
4	Entertainment	138276
5	Lifestyle	118331
6	Books & Reference	116728
7	Personalization	89210
8	Health & Fitness	83510
9	Productivity	79698

```
In [49]: top_10_category = top_category.head(10)
categories = top_10_category['Category']
count = top_10_category['count']

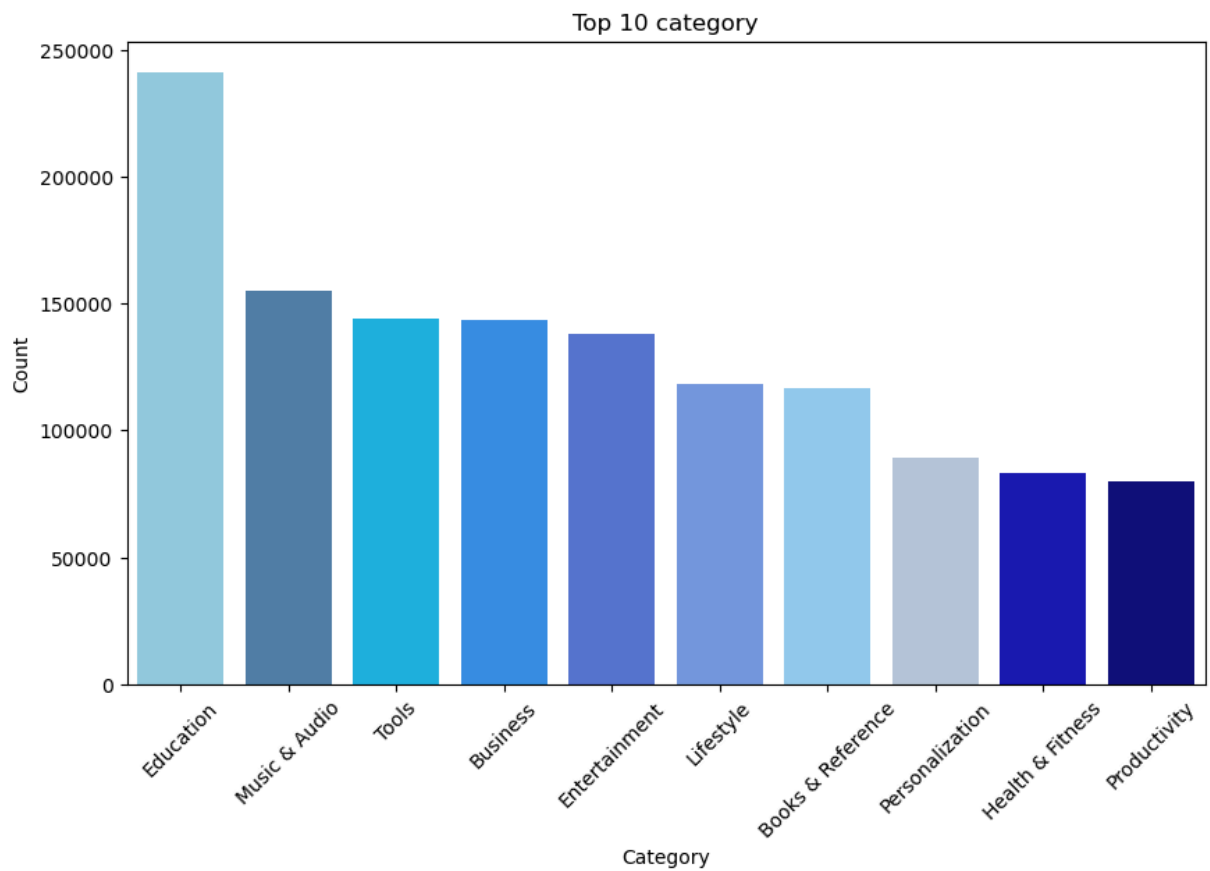
plt.figure(figsize=(10,6))
cols = ['skyblue', 'steelblue', 'deepskyblue', 'dodgerblue', 'royalblue',
        'cornflowerblue', 'lightskyblue', 'lightsteelblue', 'mediumblue', 'darkbl

sns.barplot(x = categories , y = count ,palette= cols)
plt.title('Top 10 category')
plt.ylabel('Count')
plt.xticks(rotation = 45)
plt.show()
```

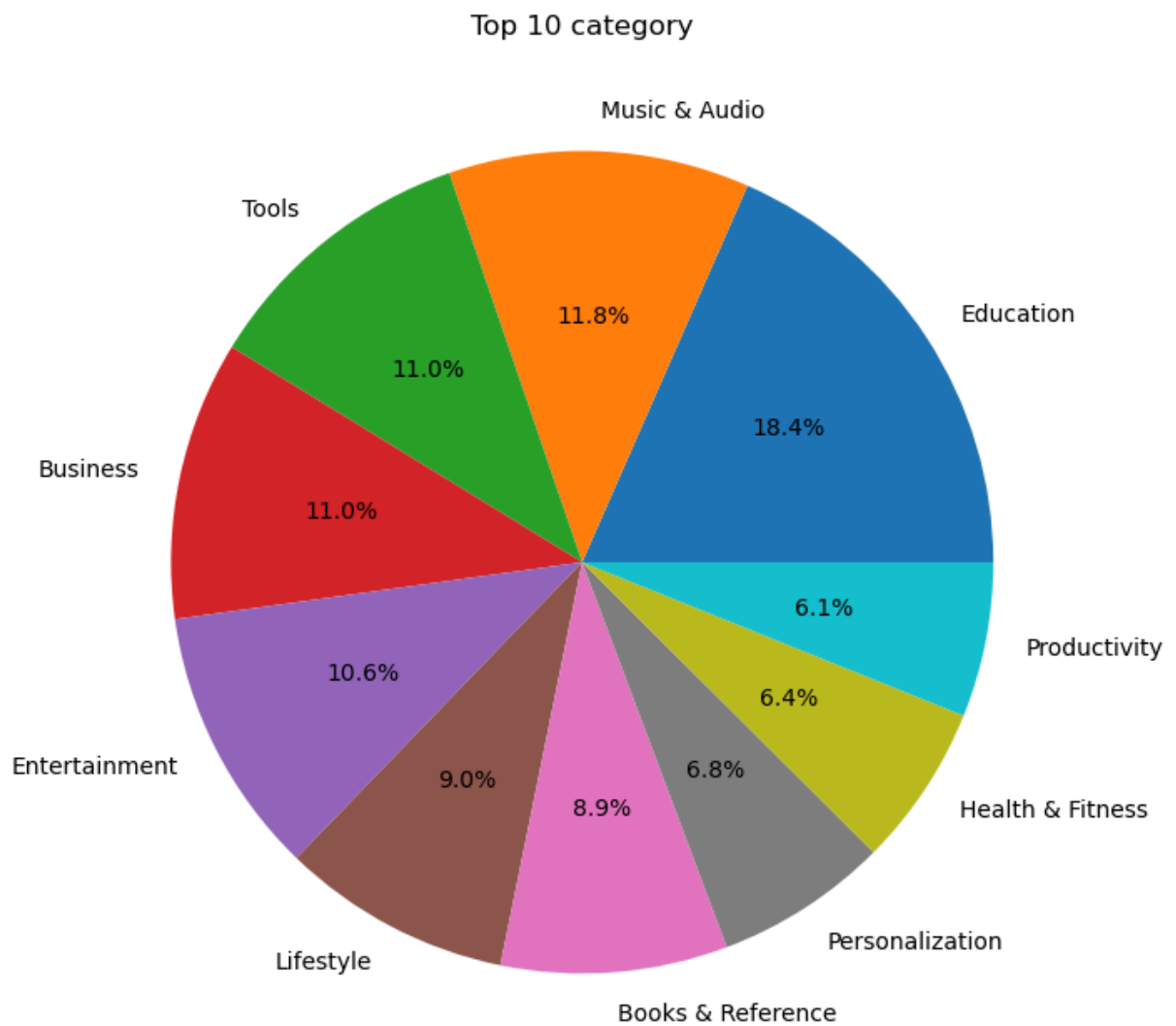
C:\Users\HP\AppData\Local\Temp\ipykernel_13168\1934875978.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x = categories , y = count ,palette= cols)
```



```
In [50]: plt.figure(figsize=(8,8))
plt.pie(top_10_category['count'],
        labels=top_10_category['Category'],
        autopct = '%1.1f%%')
plt.title('Top 10 category')
plt.show()
```



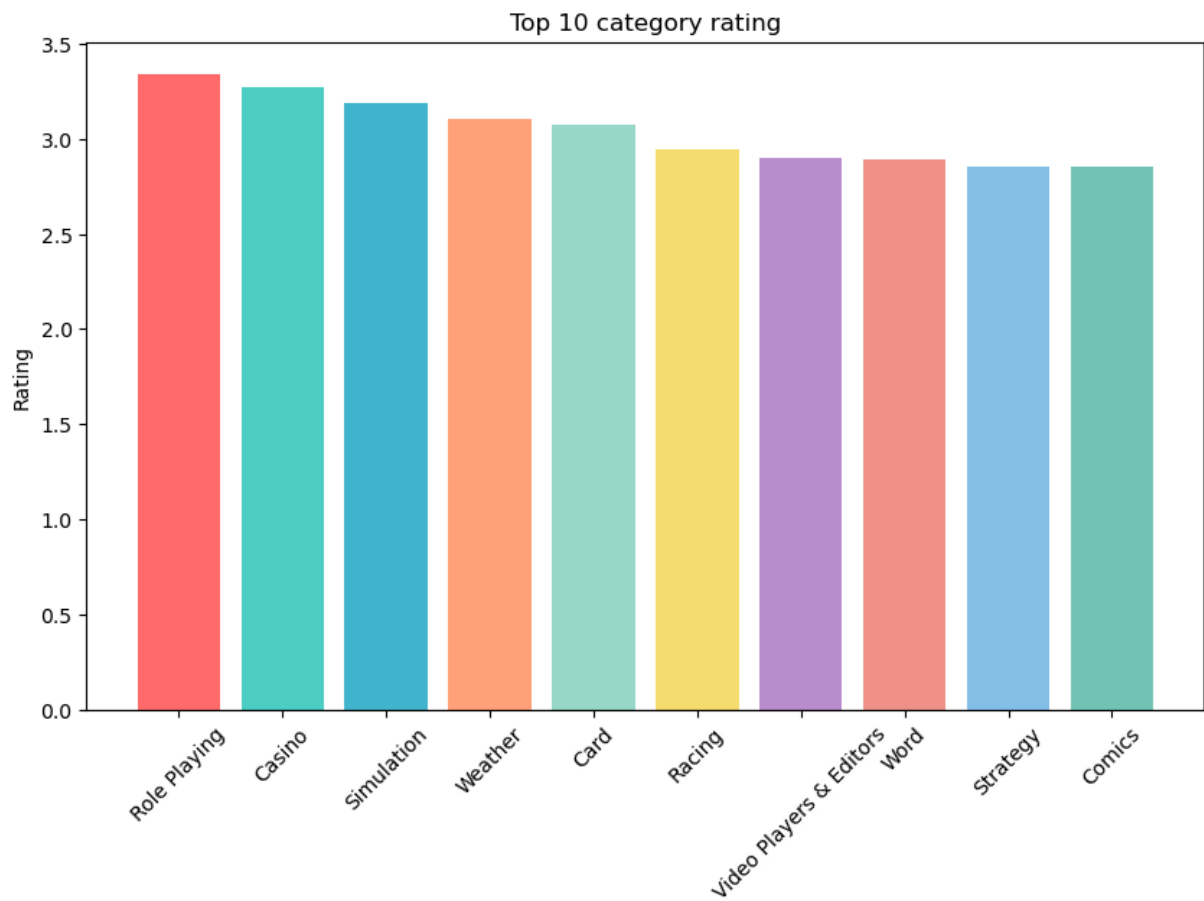
- Q2.In what category are the top ten highest ratings?

```
In [51]: top_category_rating = df.groupby('Category')['Rating'].mean().reset_index()
top_category_rating = top_category_rating.sort_values(by='Rating' , ascending = False)
top_category_rating.head(10)
```

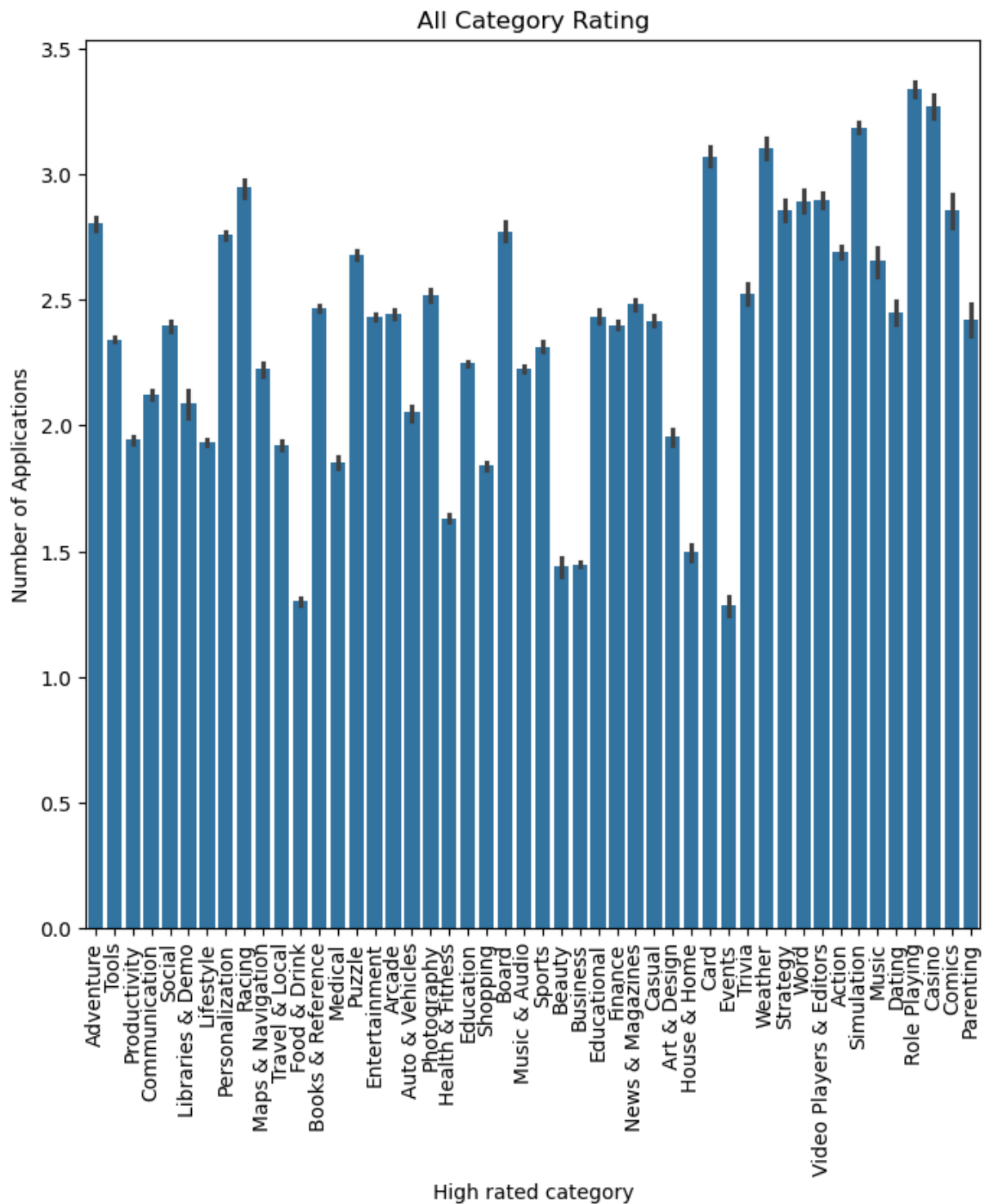
Out[51]:

	Category	Rating
36	Role Playing	3.340112
10	Casino	3.270987
38	Simulation	3.186735
46	Weather	3.105612
9	Card	3.070818
35	Racing	2.946939
45	Video Players & Editors	2.899342
47	Word	2.893573
41	Strategy	2.855995
12	Comics	2.855426

```
In [52]: plt.figure(figsize=(10,6))
cols = [
    '#FF6B6B', # red coral
    '#4ECDC4', # teal
    '#45B7D1', # sky blue
    '#FFA07A', # light salmon
    '#98D8C8', # mint
    '#F7DC6F', # yellow
    '#BB8FCE', # lavender
    '#F1948A', # soft red
    '#85C1E9', # light blue
    '#73C6B6'  # aqua green
]
plt.bar(top_category_rating['Category'], top_category_rating['Rating'],color= cols)
plt.title('Top 10 category rating')
plt.ylabel('Rating')
plt.xticks(rotation = 45)
plt.show()
```



```
In [53]: plt.figure(figsize=(8,8))
plt.xticks(rotation = 90)
plt.xlabel('High rated category')
plt.ylabel('Number of Applications')
plt.title('All Category Rating')
sns.barplot(x = df.Category , y = df.Rating)
plt.show()
```



```
In [54]: df['Content Rating'].unique()
```

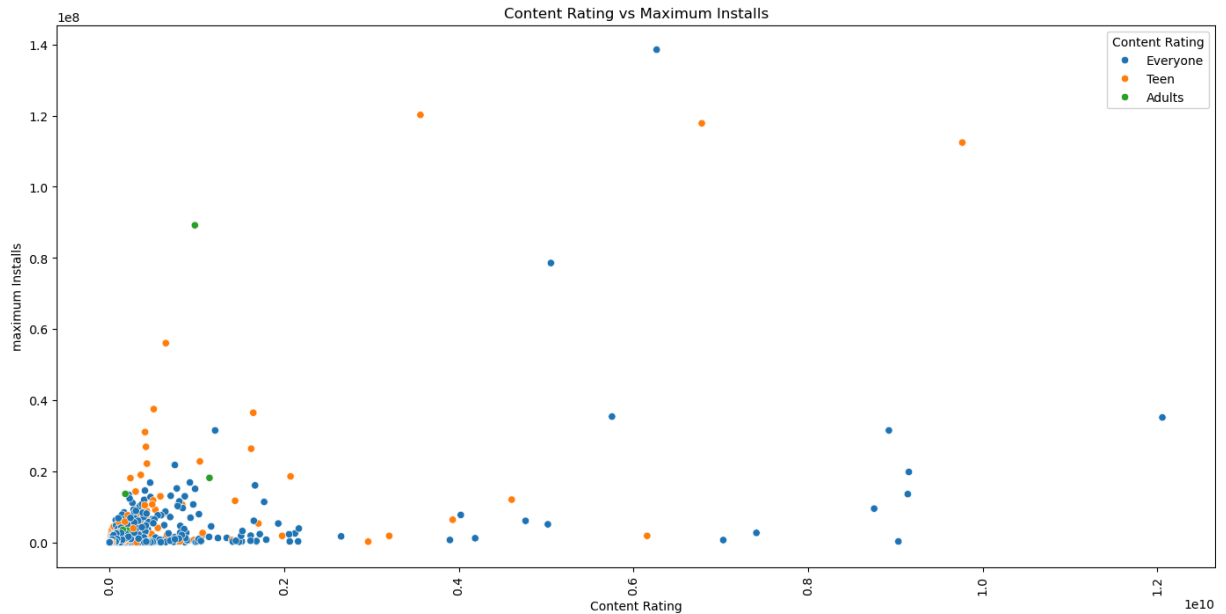
```
Out[54]: array(['Everyone', 'Teen', 'Adults'], dtype=object)
```

```
In [55]: plt.figure(figsize=(17,8))
plt.xticks(rotation = 90)
plt.xlabel('Content Rating')
plt.ylabel('maximum Installs')
plt.title('Content Rating vs Maximum Installs')
```

```
sns.scatterplot(data = df ,x = "Maximum Installs" , y = "Rating Count", hue = "Content Rating")
plt.show()
```

C:\Users\HP\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```
fig.canvas.print_figure(bytes_io, **kw)
```



- Q3. Which Play Store categories have the most Installs, based on the count?

```
In [56]: df['Category'].unique()
```

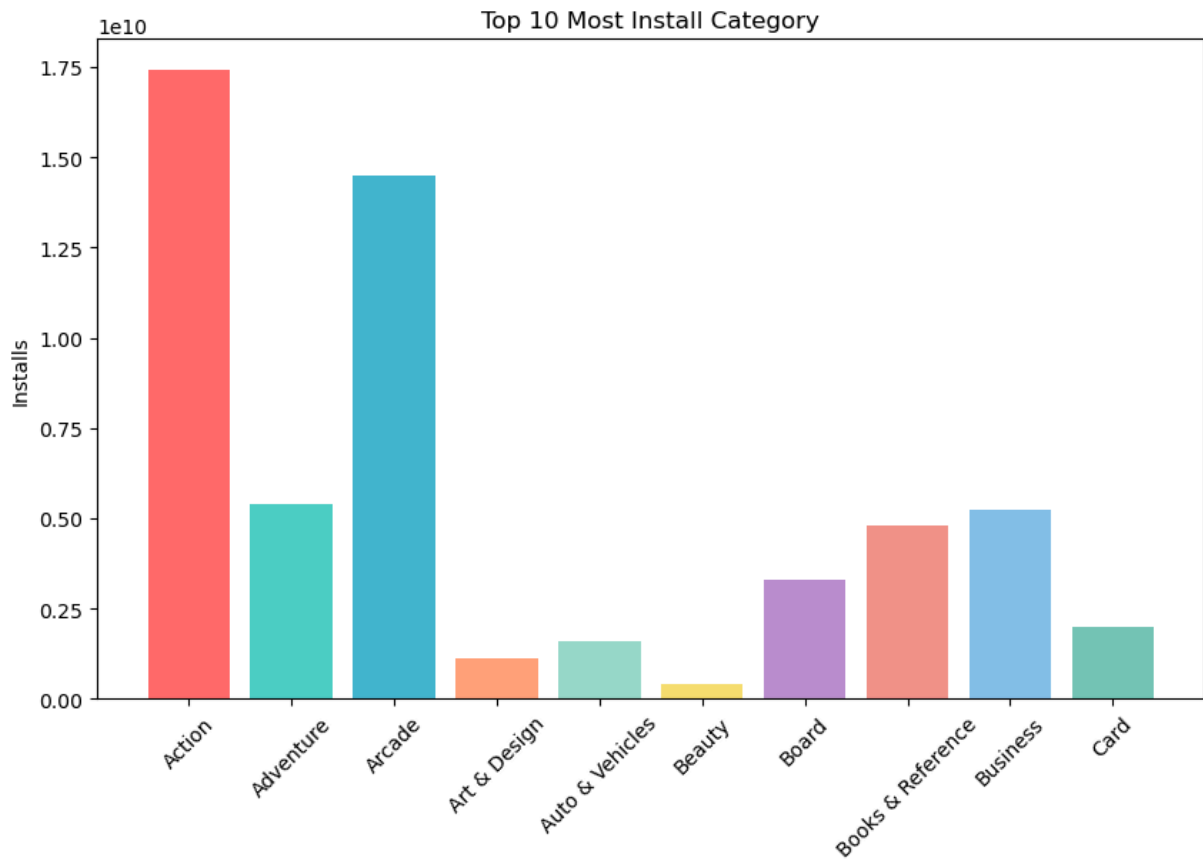
```
Out[56]: array(['Adventure', 'Tools', 'Productivity', 'Communication', 'Social',
               'Libraries & Demo', 'Lifestyle', 'Personalization', 'Racing',
               'Maps & Navigation', 'Travel & Local', 'Food & Drink',
               'Books & Reference', 'Medical', 'Puzzle', 'Entertainment',
               'Arcade', 'Auto & Vehicles', 'Photography', 'Health & Fitness',
               'Education', 'Shopping', 'Board', 'Music & Audio', 'Sports',
               'Beauty', 'Business', 'Educational', 'Finance', 'News & Magazines',
               'Casual', 'Art & Design', 'House & Home', 'Card', 'Events',
               'Trivia', 'Weather', 'Strategy', 'Word', 'Video Players & Editors',
               'Action', 'Simulation', 'Music', 'Dating', 'Role Playing',
               'Casino', 'Comics', 'Parenting'], dtype=object)
```

```
In [63]: Category_installs = df.groupby('Category')['Installs'].sum().reset_index()
top_10_installs = Category_installs.head(10)
top_10_installs
```

Out[63]:

	Category	Installs
0	Action	1.740038e+10
1	Adventure	5.390120e+09
2	Arcade	1.450133e+10
3	Art & Design	1.116399e+09
4	Auto & Vehicles	1.594746e+09
5	Beauty	4.043288e+08
6	Board	3.290827e+09
7	Books & Reference	4.820201e+09
8	Business	5.236670e+09
9	Card	1.989329e+09

```
In [64]: plt.figure(figsize=(10,6))
cols = [
    '#FF6B6B', # red coral
    '#4ECDC4', # teal
    '#45B7D1', # sky blue
    '#FFA07A', # light salmon
    '#98D8C8', # mint
    '#F7DC6F', # yellow
    '#BB8FCE', # lavender
    '#F1948A', # soft red
    '#85C1E9', # light blue
    '#73C6B6'  # aqua green
]
plt.bar(top_10_installs['Category'] , top_10_installs['Installs'],color= cols)
plt.title('Top 10 Most Install Category')
plt.ylabel('Installs')
plt.xticks(rotation = 45)
plt.show()
```

```
In [65]: top_category_installs = pd.merge(top_category ,top_10_installs , on = 'Category' )
top_category_installs.head(10)
```

```
Out[65]:
```

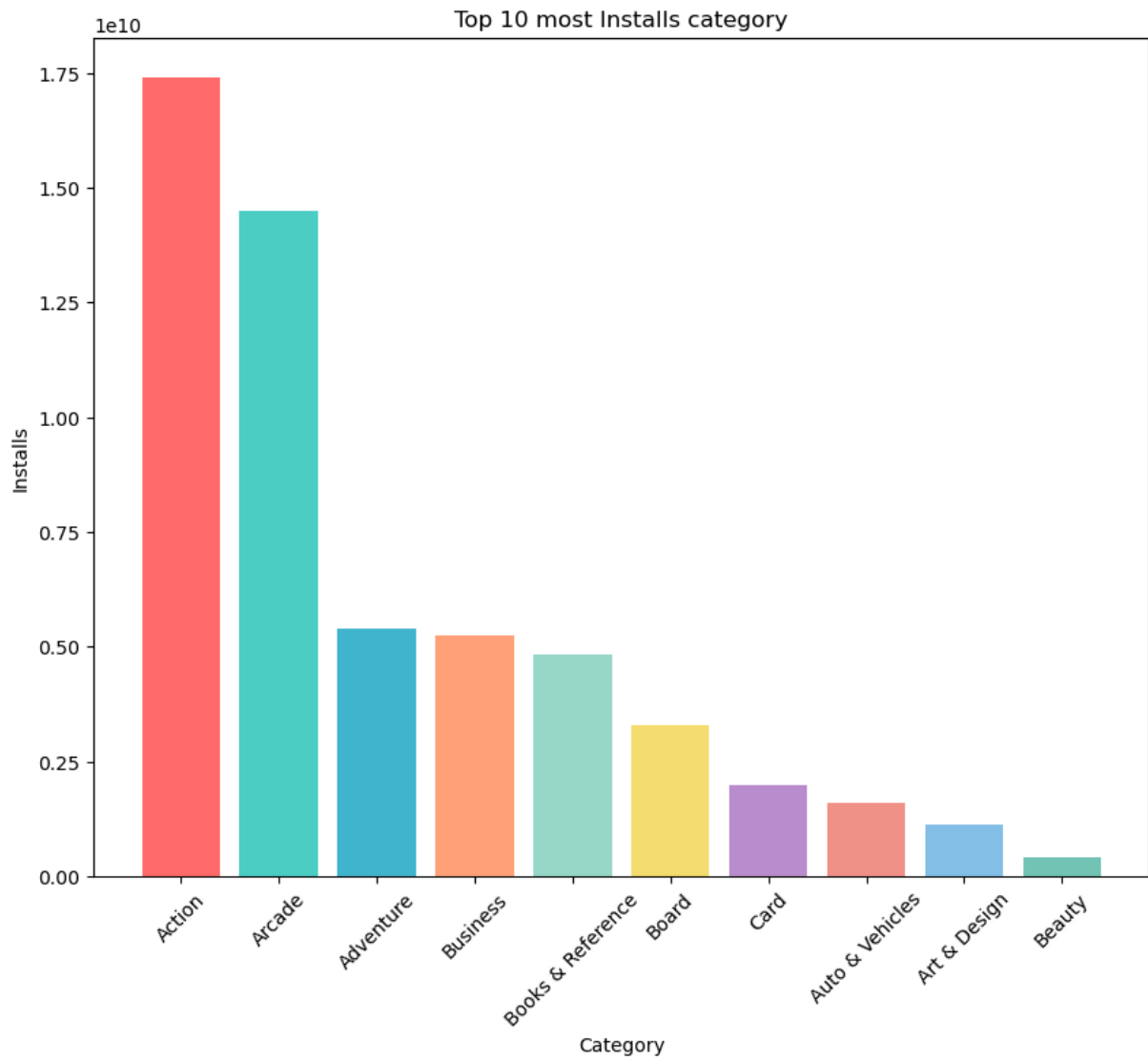
	Category	count	Installs
0	Business	143771	5.236670e+09
1	Books & Reference	116728	4.820201e+09
2	Arcade	53792	1.450133e+10
3	Action	27555	1.740038e+10
4	Adventure	23203	5.390120e+09
5	Art & Design	18539	1.116399e+09
6	Auto & Vehicles	18280	1.594746e+09
7	Beauty	11772	4.043288e+08
8	Board	10588	3.290827e+09
9	Card	8179	1.989329e+09

```
In [66]: top_category_installs = top_category_installs.sort_values(by='Installs' , ascending
top_10_installs)
```

Out[66]:

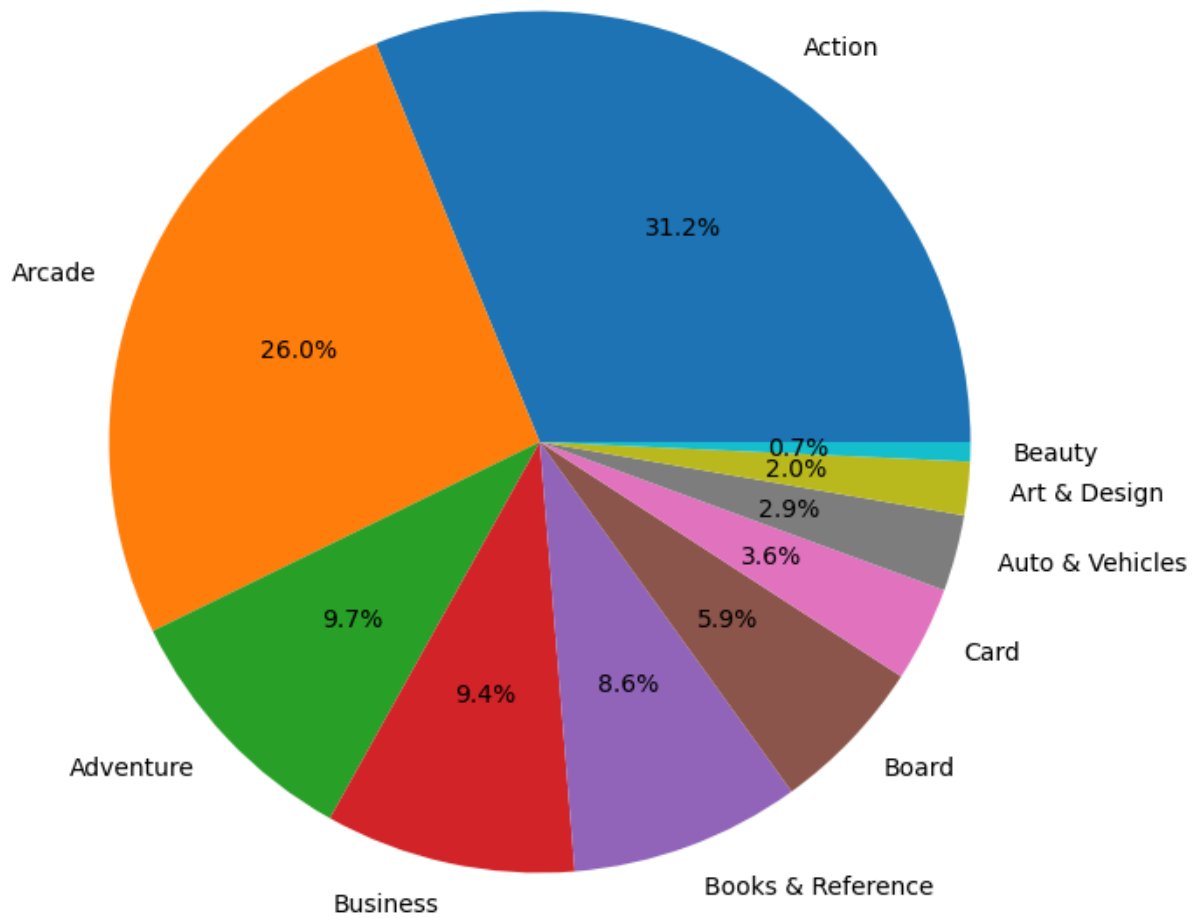
	Category	Installs
0	Action	1.740038e+10
1	Adventure	5.390120e+09
2	Arcade	1.450133e+10
3	Art & Design	1.116399e+09
4	Auto & Vehicles	1.594746e+09
5	Beauty	4.043288e+08
6	Board	3.290827e+09
7	Books & Reference	4.820201e+09
8	Business	5.236670e+09
9	Card	1.989329e+09

```
In [69]: plt.figure(figsize=(10,8))
plt.title('Top 10 most Installs category')
plt.xlabel('Category')
plt.ylabel('Installs')
plt.bar(top_category_installs['Category'],top_category_installs['Installs'], color
plt.xticks(rotation = 45)
plt.show()
```



```
In [70]: plt.figure(figsize=(8,8))
plt.pie(top_category_installs['Installs'],
        labels=top_category_installs['Category'],
        autopct = '%1.1f%%')
plt.title('Top 10 Most Installs Category')
plt.show()
```

Top 10 Most Installs Category



- Q4. What category has the most popular free and paid apps?

```
In [71]: df.columns
```

```
Out[71]: Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',  
              'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',  
              'Size', 'Minimum Android', 'Developer Id', 'Developer Website',  
              'Developer Email', 'Released', 'Last Updated', 'Content Rating',  
              'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',  
              'Scraped Time', 'Rating Category'],  
             dtype='object')
```

```
In [72]: df['Free'].unique()
```

```
Out[72]: array([ True, False])
```

```
In [74]: df['Price'].unique()
```

```
Out[74]: array([ 0.          ,  1.99          ,  4.99          , ...,  3.041816, 26.746362,
                18.903596])
```

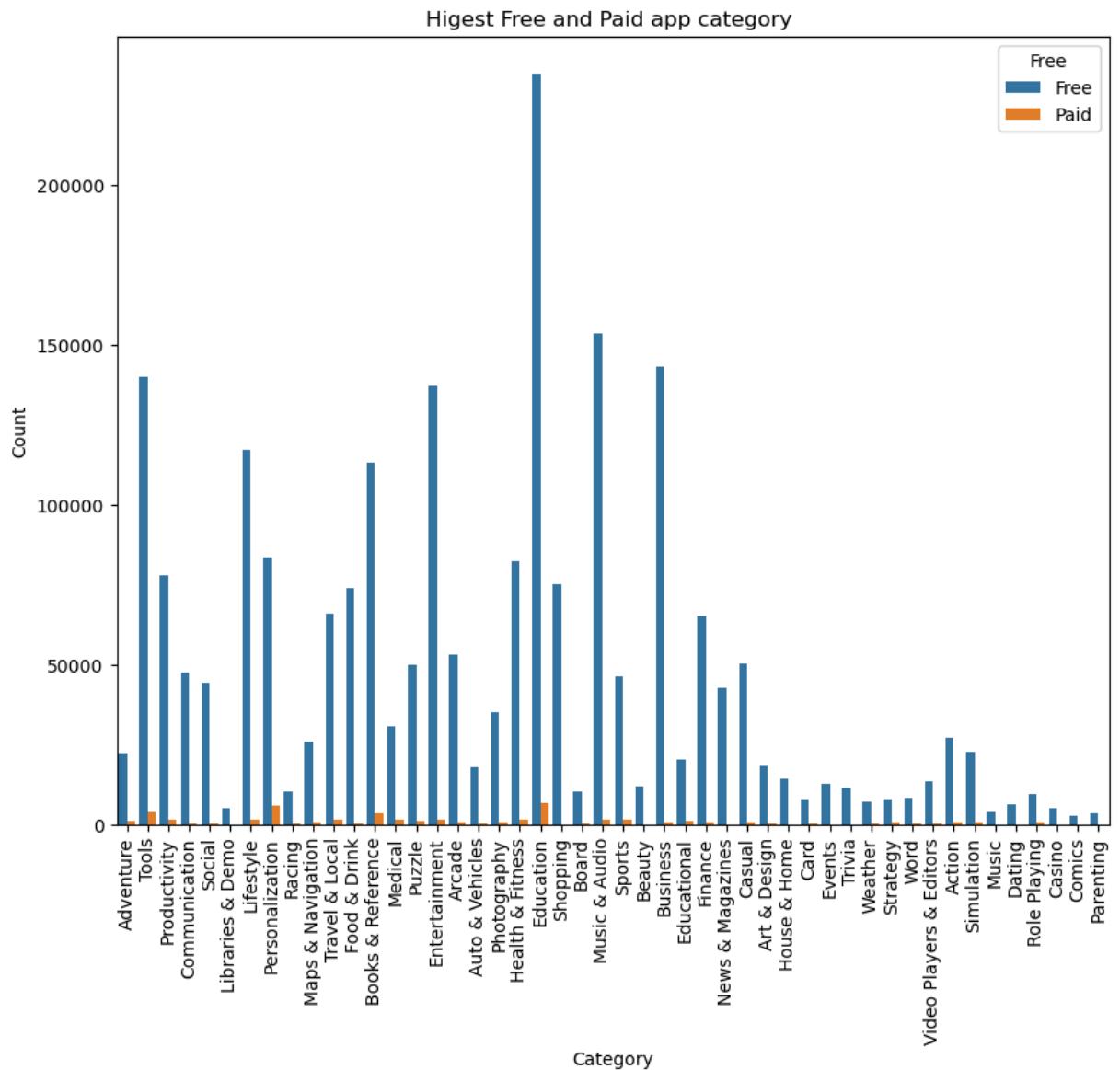
```
In [76]: df.groupby('Category')['Price'].value_counts().reset_index(name = 'Count').sort_val
```

```
Out[76]:
```

	Category	Price	Count
806	Education	0.0	234543
2038	Music & Audio	0.0	153382
481	Business	0.0	143101
2936	Tools	0.0	139911
1192	Entertainment	0.0	136894
1585	Lifestyle	0.0	116989
292	Books & Reference	0.0	113131
2189	Personalization	0.0	83443
1422	Health & Fitness	0.0	82097
2348	Productivity	0.0	77967

```
In [88]: df['Free'] = df['Price'] == 0
df['Free'] = df['Free'].replace({True:'Free' , False : 'Paid'})
# df['Free']
```

```
In [89]: plt.figure(figsize=(10,8))
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Higest Free and Paid app category ')
plt.xticks(rotation = 90)
sns.countplot(x='Category' , data = df, hue='Free')
plt.show()
```



```
In [86]: df['Free'].value_counts()
```

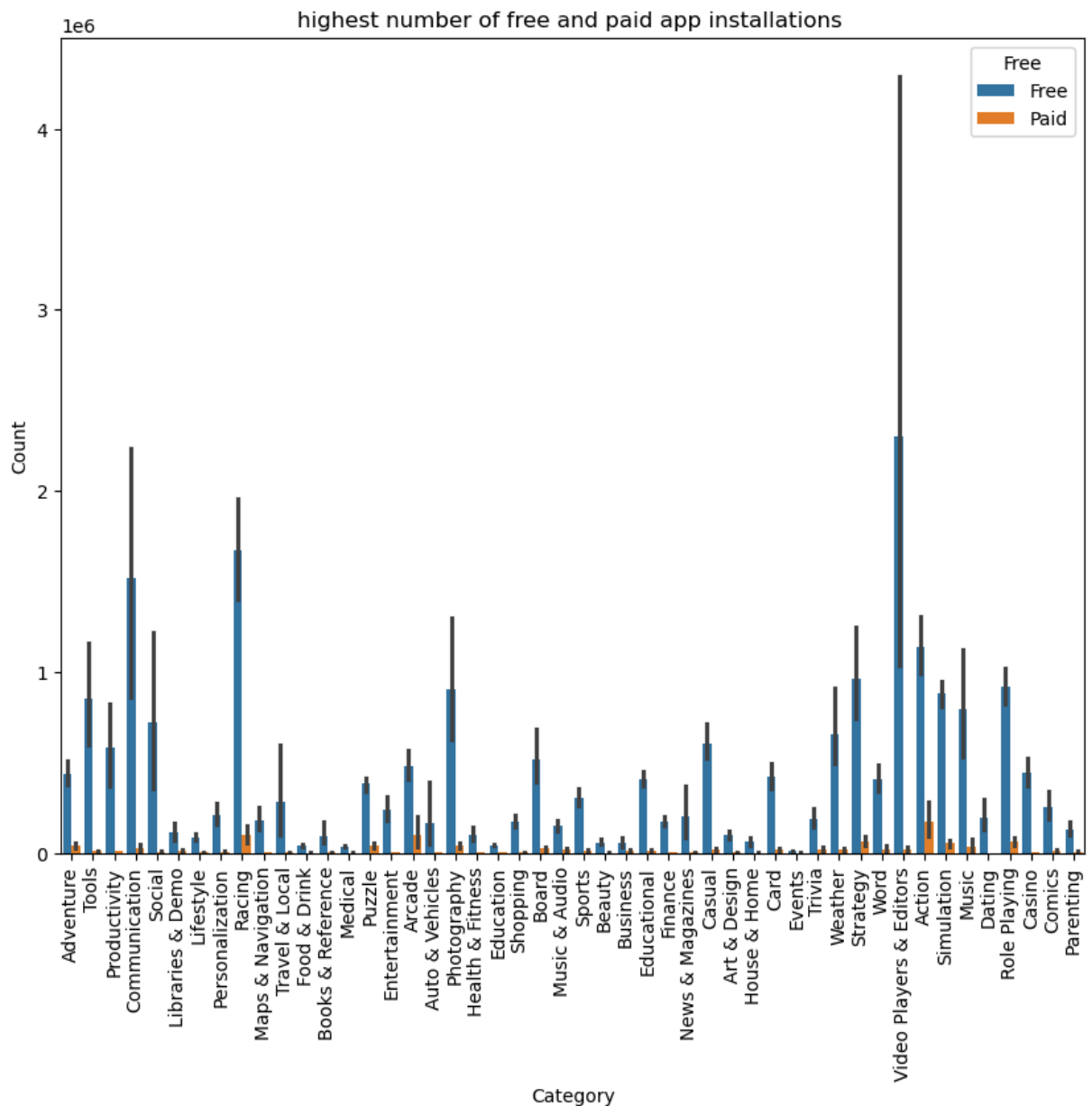
```
Out[86]: Free
0.000000    2268011
0.990000     11851
1.990000     5817
2.990000     3921
1.490000     3823
...
109.450000      1
4.870000        1
54.402466        1
7.852035         1
2.820000         1
Name: count, Length: 1063, dtype: int64
```

```
In [90]: df['Free'].unique()
```

```
Out[90]: array(['Free', 'Paid'], dtype=object)
```

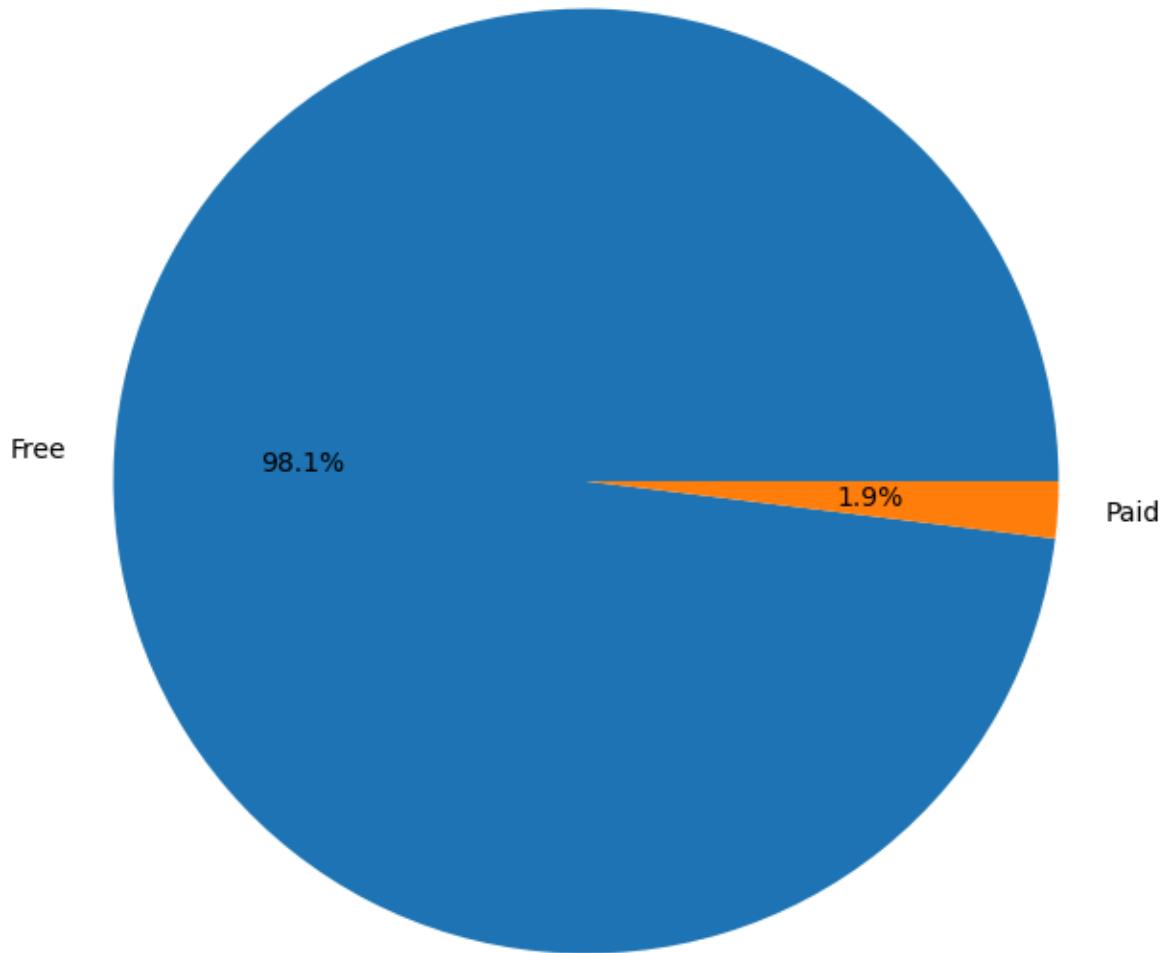
- Q4.What category has the highest number of free and paid app installations?

```
In [95]: plt.figure(figsize=(10,8))
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('highest number of free and paid app installations ')
plt.xticks(rotation = 90)
sns.barplot(x='Category' , y = 'Maximum Installs', data = df, hue='Free')
plt.show()
```



```
In [98]: plt.figure(figsize=(8,8))
plt.pie(df['Free'].value_counts(),
        labels=['Free' , 'Paid'],
        autopct = '%1.1f%%')
plt.title('highest number of free and paid app installations')
plt.show()
```

highest number of free and paid app installations



Conclusion

In this project, we performed a comprehensive Exploratory Data Analysis (EDA) on the app dataset to understand patterns, trends, and relationships between different features.

We started with data cleaning and preprocessing, handling missing values, and converting inconsistent data formats. Outliers were identified and treated using IQR and log transformation techniques, which helped in improving data distribution.

Through visualization, we observed that most applications are free, and only a small portion are paid. Categories like gaming and tools tend to have higher installs, while rating count shows a strong relationship with installs but not always with ratings.

The analysis also revealed that the dataset is highly skewed, making transformation techniques essential for better interpretation. Correlation analysis helped in identifying important relationships between features.

Overall, this EDA provided valuable insights into user behavior, app popularity, and market trends, which can be useful for decision-making and building predictive models in the future.

In []: